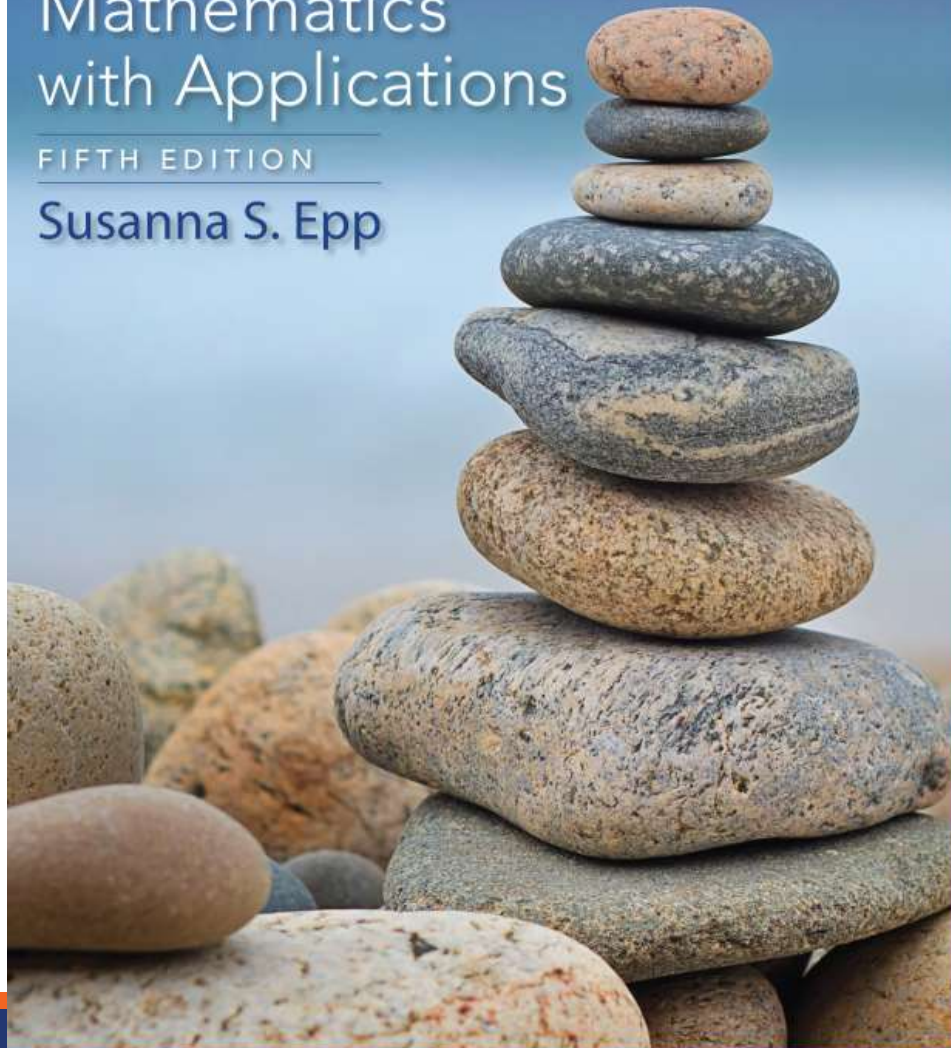


Discrete Mathematics with Applications

FIFTH EDITION

Susanna S. Epp





Faculty of Computer Science

UFCFFS-30-1– Foundations of Computing: REGULAR EXPRESSIONS AND FINITE-STATE AUTOMATA

□ Instructor: Duc Minh Vu

Learning Objectives

1. Formal Languages and Regular Expressions
2. Finite-State Automata
3. Simplifying Finite-State Automata

Reading

- ❑ Chapter 12 - Discrete Mathematics with Applications 5th Edition
(Susanna's Book)
- ❑ Chapter 13 – Discrete Mathematics and Its Applications 7th Edition
(Kenneth Rosen's book)



Formal Languages and Regular Expressions

12.1 Formal Languages and Regular Expressions

- ❑ An English sentence can be regarded as a string of words, and an English word can be regarded as a string of letters
- ❑ Computer languages are similar to English in that certain strings of characters are legitimate words of the language and certain strings of words can be put together according to certain rules to form syntactically correct programs
- ❑ A compiler for a computer language analyzes the stream of characters in a program—first to recognize individual word and sentence units, then to analyze the syntax, or grammar, of the sentences, and finally to translate the sentences into machine code.

Formal Languages and Regular Expressions

□ **Note** A Greek sigma is the traditional symbol used to denote an alphabet in a formal language. It is smaller than the Greek sigma used for a summation

Alphabet Σ:	a finite set of characters
String over Σ:	(1) a finite juxtaposition of elements (called characters) of Σ or (2) the null string λ
Length of a string over Σ:	the number of characters that made up the string, with the null string having length 0
Formal language over Σ:	a set of strings over the alphabet

Example 12.1.1 - Examples of Formal Languages

Examples of Formal Languages

Let the alphabet $\Sigma = \{a, b\}$.

- a. Define a language L_1 over Σ to be the set of all strings that begin with the character a and have length at most three characters. Find L_1 .
- b. A **palindrome** is a string that looks the same if the order of its characters is reversed. For instance, aba and $baab$ are palindromes. Define a language L_2 over Σ to be the set of all palindromes obtained using the characters of Σ . Write ten elements of L_2 .

Examples of Formal Languages

Let the alphabet $\Sigma = \{a, b\}$.

- Define a language L_1 over Σ to be the set of all strings that begin with the character a and have length at most three characters. Find L_1 .
- A **palindrome** is a string that looks the same if the order of its characters is reversed. For instance, aba and $baab$ are palindromes. Define a language L_2 over Σ to be the set of all palindromes obtained using the characters of Σ . Write ten elements of L_2 .

Solution

- $L_1 = \{a, aa, ab, aaa, aab, aba, abb\}$
- L_2 contains the following ten strings (among infinitely many others):
 $\lambda, a, b, aa, bb, aaa, bab, abba, babaabab, abaabbbbbaaba$



Notation

Let Σ be an alphabet. For each nonnegative integer n , let

Σ^n = the set of all strings over Σ that have length n ,

Σ^+ = the set of all strings over Σ that have length at least 1, and

Σ^* = the set of all strings over Σ .

Note that Σ^n is essentially the Cartesian product of n copies of Σ . The language Σ^* is called the **Kleene closure of Σ** , in honor of Stephen C. Kleene (pronounced CLAY-nee). Σ^+ is the set of all strings over Σ except for λ and is called the **positive closure of Σ** .

Example 12.1.2 The Languages Σ^n , Σ^+ , and Σ^*

Let $\Sigma = \{a, b\}$.

- Find Σ^0 , Σ^1 , Σ^2 , and Σ^3 .
- Let $A = \Sigma^0 \cup \Sigma^1$ and $B = \Sigma^2 \cup \Sigma^3$. Use words to describe A , B , and $A \cup B$.
- Describe a systematic way of writing the elements of Σ^+ . What change needs to be made to obtain the elements of Σ^* ?

Solution

- a. $\Sigma^0 = \{\lambda\}$, $\Sigma^1 = \{a, b\}$, $\Sigma^2 = \{aa, ab, ba, bb\}$, and $\Sigma^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$
- b. A is the set of all strings over Σ of length at most 1.
 B is the set of all strings over Σ of length 2 or 3.
 $A \cup B$ is the set of all strings over Σ of length at most 3.
- c. Elements of Σ^+ can be written systematically by writing all the strings of length 1, then all the strings of length 2, and so forth.

Σ^+ : $a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, aaaa, \dots$

Of course, the process of writing the strings in Σ^+ would continue forever, because Σ^+ is an infinite set. The only change that needs to be made to obtain Σ^* is to place the null string at the beginning of the list. 

Definition

Let Σ be an alphabet. Given any strings x and y over Σ , the **concatenation of x and y** is the string obtained by writing all the characters of x followed by all the characters of y . For any languages L and L' over Σ , three new languages can be defined as follows:

The **concatenation of L and L'** , denoted LL' , is

$$LL' = \{xy \mid x \in L \text{ and } y \in L'\}.$$

The **union of L and L'** , denoted $L \cup L'$, is

$$L \cup L' = \{x \mid x \in L \text{ or } x \in L'\}.$$

The **Kleene closure of L** , denoted L^* , is

$$L^* = \{x \mid x \text{ is a concatenation of any finite number of strings in } L\}.$$

Note that λ is in L^* because it is regarded as a concatenation of zero strings in L .

New Languages from Old

Let L_1 be the set of all strings consisting of an even number of a 's (namely, λ , aa , $aaaa$, $aaaaaa$, $\dots$), and let $L_2 = \{b, bb, bbb\}$. Find L_1L_2 , $L_1 \cup L_2$, and $(L_1 \cup L_2)^*$. Note that the null string λ is in L_1 because 0 is an even number.

Solution

L_1L_2 = the set of all strings that consist of an even number of a 's followed by b or by bb or by bbb .

$L_1 \cup L_2$ = the set that includes the strings b , bb , bbb and any strings consisting of an even number of a 's.

$(L_1 \cup L_2)^*$ = the set of all strings of a 's and b 's in which every occurrence of a is in a block consisting of an even number of a 's. 



The Language Defined by a Regular Expression

Definition

Given an alphabet Σ , the following are **regular expressions over Σ** :

- I. Base: $\emptyset$, λ , and each individual symbol in Σ are regular expressions over Σ .
- II. Recursion: If r and s are regular expressions over Σ , then the following are also regular expressions over Σ :
 - (i) (rs) (ii) $(r \mid s)$ (iii) (r^*) ,

where rs denotes the concatenation of r and s , r^* denotes the concatenation of r with itself any finite number (including zero) of times, and $r \mid s$ denotes either one of the strings r or s . The regular expression r^* is called the **Kleene closure** of r .
- III. Restriction: Nothing is a regular expression over Σ except for objects defined in (I) and (II) above.

As an example, one regular expression over $\Sigma = \{a, b, c\}$ is

$$a \mid (b \mid c)^* \mid (ab)^*.$$

If the alphabet Σ happens to include symbols—such as $($, or $|$, or $)$, or $*$ —special provisions have to be made to avoid ambiguity. An *escape character*, usually a backslash, is added before the potentially ambiguous symbol. For instance, a left parenthesis would be written as $\backslash($ and the backslash itself would be written as $\\$.

To eliminate parentheses, an **order of precedence** for the operations used to define regular expressions has been introduced. The highest is $*$, concatenation is next, and $|$ is the lowest. It is also customary to eliminate the outer set of parentheses in a regular expression when the order of precedence rules are sufficient for avoiding ambiguity. Thus

$$(a((bc)^*)) = a(bc)^* \quad \text{and} \quad (a \mid (bc)) = a \mid bc.$$

Example 12.1.5

Order of Precedence for the Operations in a Regular Expression

- Add parentheses to emphasize the order of precedence in the following expression:
 $ab^* \mid b^*a$.
- Use the rules about order of precedence to eliminate the parentheses in the following expression: $((a \mid ((b^*)c))(a^*))$.

Solution

- $((a(b^*)) \mid ((b^*)a))$
- $(a \mid b^*c)a^*$

Definition

For any finite alphabet Σ , the function L that associates a language to each regular expression over Σ is defined by (I)–(III) below. For each such regular expression r , $L(r)$ is called the **language defined by r** .

- I. Base: $L(\emptyset) = \emptyset$, $L(\lambda) = \{\lambda\}$, $L(a) = \{a\}$ for every a in Σ .
- II. Recursion: If $L(r)$ and $L(r')$ are the languages defined by the regular expressions r and r' over Σ , then
 - (i) $L(rr') = L(r)L(r')$ (ii) $L(r \mid r') = L(r) \cup L(r')$ (iii) $L(r^*) = (L(r))^*$.
- III. Restriction: The function L is completely determined by I and II above.



Example 12.1.7

The Language Defined by a Regular Expression

Let $\Sigma = \{0, 1\}$. Use words to describe the languages defined by the following regular expressions over Σ .

a. $0^*1^* \mid 1^*0^*$

b. $0(0 \mid 1)^*$

Solution

- a. The strings in this language consist either of a string of 0's followed by a string of 1's or of a string of 1's followed by a string of 0's. However, in either case the strings could be empty, which means that λ is also in the language.
- b. The strings in this language have to start with a 0. The 0 may be followed by any finite number (including zero) of 0's and 1's in any order. Thus the language is the set of all strings of 0's and 1's that start with a 0.

Example 12.1.8

Individual Strings in the Language Defined by a Regular Expression

In each of (a) and (b), let $\Sigma = \{a, b\}$ and consider the language L over Σ defined by the given regular expression.

- The regular expression is $a^*b(a \mid b)^*$. Write five strings that belong to L .
- The regular expression is $a^* \mid (ab)^*$. Indicate which of the following strings belong to L :

$a \quad b \quad aaaa \quad abba \quad ababab$

Solution

- The strings b , ab , $abbb$, $abaaa$, and $ababba$ are five strings from the infinitely many in L .
- The following strings are the only ones listed that belong to L : a , $aaaa$, and $ababab$. The string b does not belong to L because it is neither a string of a 's nor a string of possibly repeated ab 's. The string $abba$ does not belong to L because any two b 's that might occur in a string of L are separated by an a . ■

Example 12.1.9

A Regular Expression That Defines a Language

Let $\Sigma = \{0, 1\}$. Find regular expressions over Σ that define the following languages:

- The language consisting of all strings of 0's and 1's that have even length and in which the 0's and 1's alternate.
- The language consisting of all strings of 0's and 1's with an even number of 1's. Such strings are said to have *even parity*.
- The language consisting of all strings of 0's and 1's that do not contain two consecutive 1's.

Solution

- a. If a string in the language starts with a 1, the pattern 10 must continue for the length of the string. If it starts with 0, the pattern 01 must continue for the length of the string. Also, the null string satisfies the condition by default. Thus an answer is

$$(10)^* \mid (01)^*.$$

- b. Some basic strings with even parity are λ , 0, and 10^*1 . Concatenation of strings with even parity also have even parity. Because such a string may start or end with a string of 0's, one answer is

$$(0 \mid 10^*1)^*.$$
$$0^*1(0 \mid 10^*1)^* \\ (0 \mid 10^*1)^*10^*$$

- c. Note that a string may end in a 1, but any other 1 must be followed immediately by a 0. Thus, it is enough to enforce the rule that a 1 must be followed by a 0, unless the 1 is at the end of the string. A regular expression satisfying these conditions is

$$(0 \mid 10)^*(\lambda \mid 1).$$

Note that a given language may be defined by more than one regular expression. For example, both

$$(a^* \mid b^*)^* \quad \text{and} \quad (a \mid b)^*$$

define the language consisting of the set of all strings of a 's and b 's.

Example 12.1.10

Deciding Whether Regular Expressions Define the Same Language

In (a) and (b), determine whether the given regular expressions define the same language. If they do, describe the language. If they do not, give an example of a string that is in one of the languages but not the other.

a. $(a \mid \lambda)^*$ and a^*

b. $0^* \mid 1^*$ and $(01)^*$

Solution

- Note that because the null string λ has no characters, when it is concatenated with any other string x , the result is just x : for every string x , $x\lambda = \lambda x = x$. Now $L((a \mid \lambda)^*)$ is the set of strings formed using a and λ in any order, and so, because $a\lambda = \lambda a = a$, this is the same as the set of strings consisting of zero or more a 's. Thus $L((a \mid \lambda)^*) = L(a^*)$.
- The two languages defined by the given regular expressions are not the same: 0101 is in the second language but not the first. 



Practical Uses of Regular Expressions (option)

$[A - C]$ stands for $(A | B | C)$

and

$[0 - 9]$ stands for $(0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9)$.

Character classes are also allowed to include more than one range of characters. For instance,

$[A - C x - z]$ stands for $(A | B | C | x | y | z)$

As an example, consider the language defined by the regular expression

$$[A - Z a - z]([A - Z a - z] \mid [0 - 9])^*.$$

The following are some strings in the language:

Account Number, z23, jsmith109, Draft2rev.

Other commonly used shorthands are

$[ABC]$ to stand for $(A \mid B \mid C)$

and a single dot

$.$ to stand for an arbitrary character.

Thus, for instance, if $\Sigma = \{A, B, C\}$, then

$A.C$ stands for $(AAC \mid ABC \mid ACC)$.

When the symbol $\wedge$ is placed at the beginning of a character class, it indicates that a character of the same type as those in the range of the class is to occur at that point in the string, except for one of the specific characters indicated after the $\wedge$ sign. For instance,

$$[\wedge D - Z][0 - 9][0 - 9]^*$$

stands for any string starting with a letter of the alphabet different from D to Z , followed by any positive number of digits from 0 to 9. Examples are $B3097$, $C0046$, and so forth. If r is a regular expression, the notation $r+$ denotes the concatenation of r with itself any positive finite number of times. In symbols,

$$r+ = rr^*.$$

For example,

$$[A - Z]^+$$

represents any nonempty string of capital letters. If r is a regular expression, then

$$r? = (\lambda \mid r).$$

That is, $r?$ denotes either zero occurrences or exactly one occurrence of r . Finally, if m and n are positive integers with $m \leq n$,

$r\{n\}$ denotes the concatenation of r with itself exactly n times,

and

$r\{m, n\}$ denotes the concatenation of r with itself anywhere from m through n times.

TEST YOURSELF

Answers to Test Yourself questions are located at the end of each section.

1. If x and y are strings, the concatenation of x and y is _____.
2. If L and L' are languages, the concatenation of L and L' is _____.
3. If L and L' are languages, the union of L and L' is _____.
4. If L is a language, the Kleene closure of L is _____.
5. The set of regular expressions over an alphabet Σ is defined recursively. The base for the definition is the statement that _____. The recursion for the definition specifies that if r and s are any regular expressions over Σ , then the following are also regular expressions in the set: _____, _____, and _____.
6. The function that associates a language to each regular expression over an alphabet Σ is defined recursively. The base for the definition is the statement that $L(\emptyset) =$ _____, $L(\lambda) =$ _____, and $L(a) =$ _____ for every a in Σ . The recursion for the definition specifies that if $L(r)$ and $L(r')$ are the languages defined by the regular expressions r and r' over Σ , then $L(rr') =$ _____, $L(r \mid r') =$ _____, and $L(r^*) =$ _____.
7. The notation $[A - C]$ is an example of a _____ and denotes the regular expression _____.
8. Use of a single dot in a regular expression stands for _____.
9. The symbol $\wedge$, placed at the beginning of a character class, indicates _____.

EXERCISE SET 12.1*

In 1 and 2, let $\Sigma = \{x, y\}$ be an alphabet.

1.
 - a. Let L_1 be the language consisting of all strings over Σ that are palindromes and have length ≤ 4 . List the elements of L_1 between braces.
 - b. Let L_2 be the language consisting of all strings over Σ that begin with an x and have length ≤ 3 . List the elements of L_2 .
2.
 - a. Let L_3 be the language consisting of all strings over Σ of length ≤ 3 in which all the x 's appear to the left of all the y 's. List the elements of L_3 between braces.
 - b. List between braces the elements of Σ^4 , the set of all strings of length 4 over Σ .
 - c. Let $A = \Sigma^1 \cup \Sigma^1$ and $B = \Sigma^3 \cup \Sigma^4$. Describe A , B , and $A \cup B$ in words.
3.
 - a. If the expression $ab + cd + \cdot$ in postfix notation is converted to infix notation, what is the result?
 - H b.** Let $\Sigma = \{1, 2, *, /\}$ and let L be the set of all strings over Σ obtained by writing first a number (1 or 2), then a second number (1 or 2), which can be the same as the first one, and finally an operation (denoted $*$ or $/$, where $*$ indicates multiplication and $/$ indicates division). Then L is a set of postfix, or reverse Polish, expressions. List all the elements of L between braces, and evaluate the resulting expressions.

In 4–6, describe L_1L_2 , $L_1 \cup L_2$, and $(L_1 \cup L_2)^*$ for the given languages L_1 and L_2 .

4. L_1 is the set of all strings of a 's and b 's that start with an a and contain only that one a ; L_2 is the set of all strings of a 's and b 's that contain an even number of a 's.
5. L_1 is the set of all strings of a 's, b 's, and c 's that contain no c 's and have the same number of a 's as b 's; L_2 is the set of all strings of a 's, b 's, and c 's that contain no a 's or b 's.
6. L_1 is the set of all strings of 0's and 1's that start with a 0; L_2 is the set of all strings of 0's and 1's that end with a 0.

In 7–9, add parentheses to emphasize the order of precedence in the given expressions.

7. $(a \mid b^*b)(a^* \mid ab)$ 8. $0^*1 \mid 0(0^*1)^*$

9. $(x \mid yz^*)^*(yx \mid (yz)^*z)$

In 10–12, use the rules about order of precedence to eliminate the parentheses in the given regular expression.

10. $((a(b^*)) \mid (c(b^*))) ((ac) \mid (bc)) \quad (ab^*|cb^*)(ac|bc)$

11. $(1(1^*)) \mid ((1(0^*)) \mid ((1^*)1))$

12. $(xy)((x^*)y)^* \mid (((yx) \mid y)(y^*))$

In 13–15, use set notation to derive the language defined by the given regular expression. Assume $\Sigma = \{a, b, c\}$.

13. $\lambda \mid ab$

14. $\emptyset \mid \lambda$

15. $(a \mid b)c$

$$L=\{\lambda, ab\}$$

$$(0^*1^*)^* = (0 \mid 1)^*$$

$$0^*1^* = \{0\dots 01\dots 1\}$$

In 16–18, write five strings that belong to the language defined by the given regular expression.

16. $0^*1(0^*1^*)^*$ **17.** $b^* \mid b^*ab^*$ **18.** $x^*(yxxxy \mid x)^*$

In 19–21, use words to describe the language defined by the given regular expression.

19. $b^*ab^*ab^*a$ **20.** $1(0 \mid 1)^*00$ **21.** $(x \mid y)y(x \mid y)^*$

In 22–24, indicate whether the given strings belong to the language defined by the given regular expression. Briefly justify your answers.

22. Expression: $(b \mid \lambda)a(a \mid b)^*a(b \mid \lambda)$, strings: aaba, baabb

23. Expression: $(x^*y \mid zy^*)^*$, strings: zyyxz, zyyzy

$zyyxz = (z)(y)(y)(x)(z)$
 $zyyzy = (zyy)(zy)$

24. Expression: $(01^*2)^*$, strings: 120, 01202

$(01^*2)^*$ e.g. 20120

In 25–27, find a regular expression that defines the given language.

- 25.** The language consisting of all strings of 0's and 1's with an odd number of 1's. (Such a string is said to have *odd parity*.) $0^*1(0^*10^*10^*)^*$

- 26.** The language consisting of all strings of a 's and b 's in which the third character from the end is a b . $(a|b)^*b(aa|bb|ab|ba)$

- 27.** The language consisting of strings of x 's and y 's in which the elements in every pair of x 's are separated by at least one y .

Let r , s , and t be regular expressions over $\Sigma = \{a, b\}$. In 28–30, determine whether the two regular expressions define the same language. If they do, describe the language. If they do not, give an example of a string that is in one of the languages but not the other.

28. $(r \mid s)t$ and $rt \mid st$

29. $(rs)^*$ and r^*s^*

30. $(rs)^*$ and $((rs)^*)^*$

In 31–39, write a regular expression to define the given set of strings. Use the shorthand notations given in the section whenever convenient. In most cases, your expression will describe other strings in addition to the given ones, but try to make your answer fit the given strings as closely as possible within reasonable space limitations.

- 31.** All words that are written in lowercase letters and start with the letters *pre* but do not consist of *pre* all by itself.
- 32.** All words that are written in uppercase letters, and contain the letters *BIO* (as a unit) or *INFO* (as a unit).
- 33.** All words that are written in lowercase letters, end in *ly*, and contain at least five letters.

- 34.** All words that are written in lowercase letters and contain at least one of the vowels a, e, i, o, or u.
- 35.** All words that are written in lowercase letters and contain exactly one of the vowels a, e, i, o, or u.
- 36.** All words that are written in uppercase letters and do not start with one of the vowels A, E, I, O, or U but contain exactly two of these vowels next to each other.
- 37.** All United States social security numbers (which consist of three digits, a hyphen, two digits, another hyphen, and finally four more digits), where the final four digits start with a 3 and end with a 6.
- 38.** All telephone numbers that have three digits, then a hyphen, then three more digits, then a hyphen, and then four digits, where the first three digits are either 800 or 888 and the last four digits start and end with a 2.



Finite-State Automata

Example 12.2.1

A Simple Vending Machine

A simple vending machine dispenses bottles of juice that cost \$1 each. The machine accepts quarters and half-dollars only and does not give change. As soon as the amount deposited equals or exceeds \$1 the machine releases a bottle of juice. The next coin deposited starts the process over again. The operation of the machine is represented by the diagram of Figure 12.2.1.

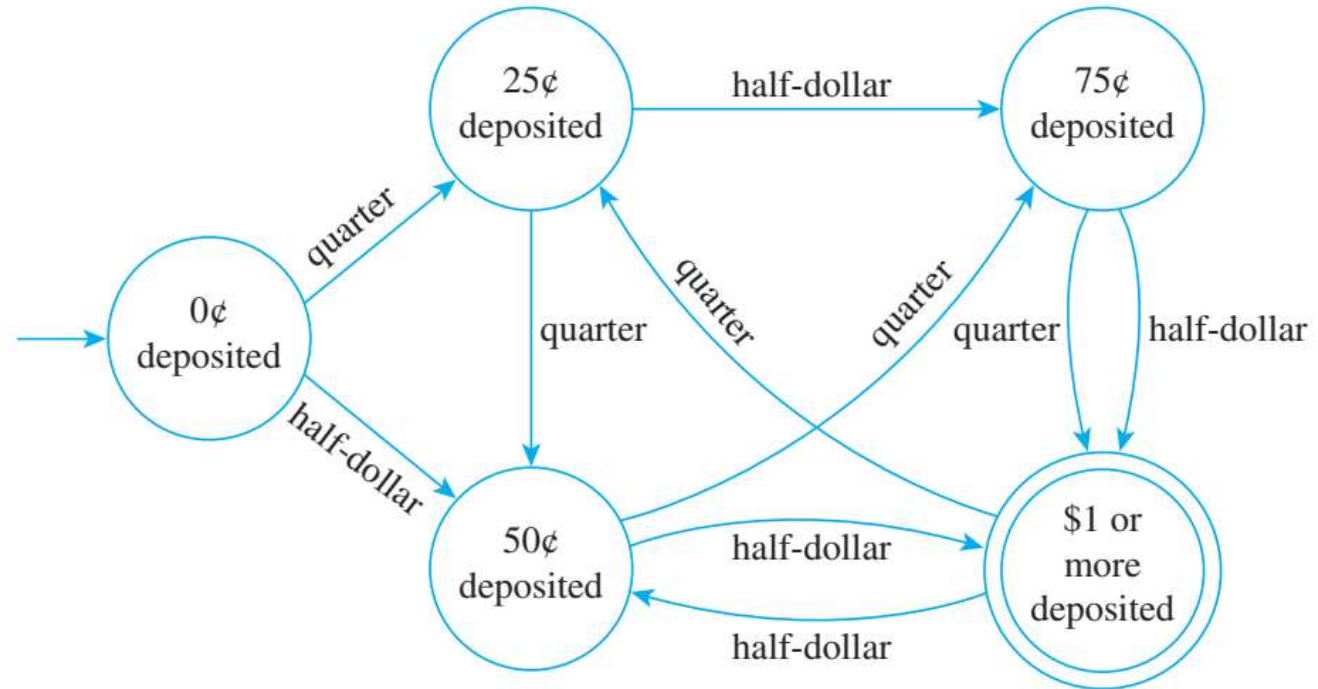


FIGURE 12.2.1 A Simple Vending Machine

Equivalently, the operation of the vending machine can be represented by a *next-state table* as shown in Table 12.2.1.

TABLE 12.2.1 Next-State Table

		Input	
		Quarter	Half-Dollar
→ State	0¢ deposited	25¢ deposited	50¢ deposited
	25¢ deposited	50¢ deposited	75¢ deposited
	50¢ deposited	75¢ deposited	\$1 or more deposited
	75¢ deposited	\$1 or more deposited	\$1 or more deposited
	⊙ \$1 or more deposited	25¢ deposited	50¢ deposited

Definition of a Finite-State Automaton

Definition

A **finite-state automaton** A consists of five objects:

1. A finite set I , called the **input alphabet**, of input symbols.
2. A finite set S of **states** the automaton can assume.
3. A designated state s_0 called the **initial state**.
4. A designated set of states called the set of **accepting states**.
5. A **next-state function** $N: S \times I \rightarrow S$ that associates a “next-state” to each ordered pair consisting of a “current state” and a “current input.” For each state s in S and input symbol m in I , $N(s, m)$ is the state to which A goes if m is input to A when A is in state s .

The operation of a finite-state automaton is commonly described by a diagram called a **(state-) transition diagram**, similar to that of Figure 12.2.1. It is called a *transition diagram* because it shows the transitions the machine makes from one state to another in response to various inputs. In a transition diagram, states are represented by circles and accepting states by double circles. There is one arrow that points to the initial state and there are other arrows that are labeled with input symbols and point from each state to other states to indicate the action of the next-state function. Specifically, an arrow from state s to state t labeled m means that $N(s, m) = t$.

The **next-state table** for an automaton shows the values of the next-state function N for all possible states s and input symbols i . In the **annotated next-state table**, the initial state is indicated by an arrow and the accepting states are marked by double circles.

Example 12.2.2 A Finite-State Automaton Given by a Transition Diagram

Consider the finite-state automaton A defined by the transition diagram shown in Figure 12.2.2.

- What are the states of A ?
- What are the input symbols of A ?
- What is the initial state of A ?
- What are the accepting states of A ?
- Determine $N(s_1, 1)$.
- Construct the annotated next-state table for A .

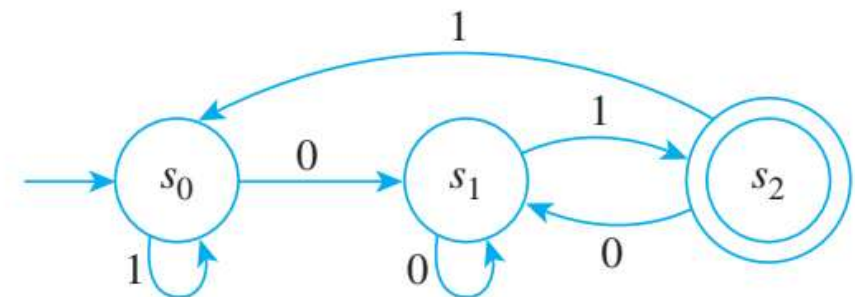


FIGURE 12.2.2

Solution

- The states of A are s_0 , s_1 , and s_2 [since these are the labels of the circles].
- The input symbols of A are 0 and 1 [since these are the labels of the arrows].
- The initial state of A is s_0 [since the unlabeled arrow points to s_0].
- The only accepting state of A is s_2 [since this is the only state marked by a double circle].
- $N(s_1, 1) = s_2$ [since there is an arrow from s_1 to s_2 labeled 1]

f.

		Input	
		0	1
State	s_0	s_1	s_0
	s_1	s_1	s_2
	s_2	s_1	s_0

Example 12.2.3

A Finite-State Automaton Given by an Annotated Next-State Table

Consider the finite-state automaton A defined by the following annotated next-state table:

- What are the states of A ?
- What are the input symbols of A ?
- What is the initial state of A ?
- What are the accepting states of A ?
- Find $N(U, c)$.
- Draw the transition diagram for A .

		Input			
		a	b	c	
State	$\rightarrow$	U	Z	Y	Y
	$\odot$	V	V	V	V
		Y	Z	V	Y
	$\odot$	Z	Z	Z	Z

Solution

- a. The states of A are U , V , Y , and Z .
- b. The input symbols of A are a , b , and c .
- c. The initial state of A is U [since the arrow points to U].
- d. The accepting states of A are V and Z [since these are marked with double circles].
- e. $N(U, c) = Y$ [since the entry in the row labeled U and the column labeled c of the next-state table is Y].
- f. The transition diagram for A is shown in Figure 12.2.3. It can be drawn more compactly by labeling arrows with multiple-input symbols where appropriate. This is illustrated in Figure 12.2.4.

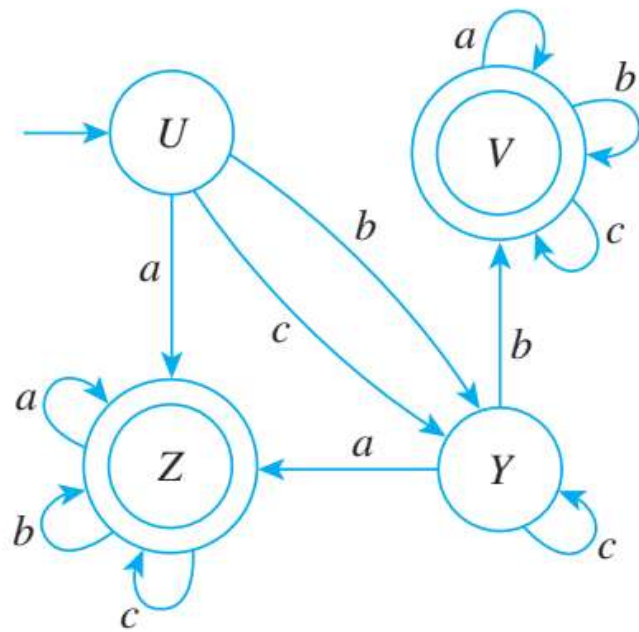


FIGURE 12.2.3

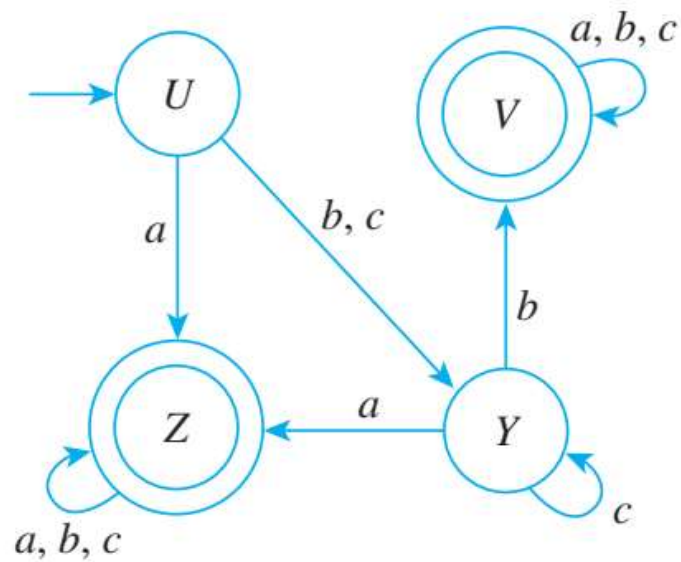


FIGURE 12.2.4



TEST YOURSELF

1. The five objects that make up a finite-state automaton are _____, _____, _____, _____, and _____.
2. The next-state table for an automaton shows the values of _____.
3. In the annotated next-state table, the initial state is indicated with an _____ and the accepting states are marked by _____.
4. A string w consisting of input symbols is accepted by a finite-state automaton A if, and only if, _____.
5. The language accepted by a finite-state automaton A is _____.
6. If N is the next-state function for a finite-state automaton A , the eventual-state function N^* is defined as follows: For each state s of A and for each string w that consists of input symbols of A , $N^*(s, w) = \underline{\hspace{2cm}}$.
7. One part of Kleene's theorem says that given any language that is accepted by a finite-state automaton, there is _____.
8. The second part of Kleene's theorem says that given any language defined by a regular expression, there is _____.
9. A regular language is _____.
10. Given the language consisting of all strings of the form $a^k b^k$, where k is a positive integer, the pigeonhole principle can be used to show that the language is _____.

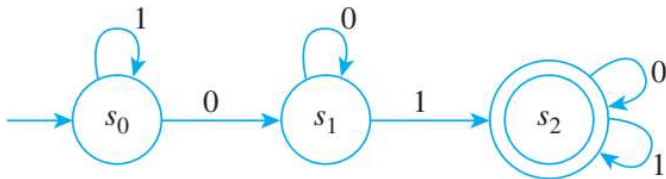
EXERCISE SET 12.2

1. Find the state of the vending machine in Example 12.2.1 after each of the following sequences of coins have been input.
 - a. Quarter, half-dollar, quarter
 - b. Quarter, half-dollar, half-dollar
 - c. Half-dollar, quarter, quarter, quarter, half-dollar

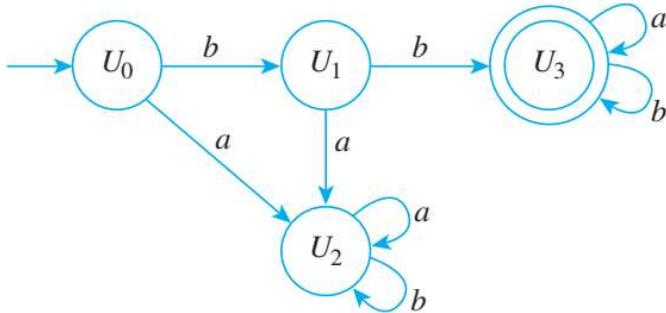
In 2–7, a finite-state automaton is given by a transition diagram. For each automaton:

- Find its states.
- Find its input symbols.
- Find its initial state.
- Find its accepting states.
- Write its annotated next-state table.

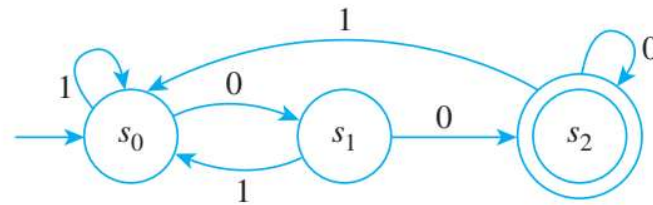
2.



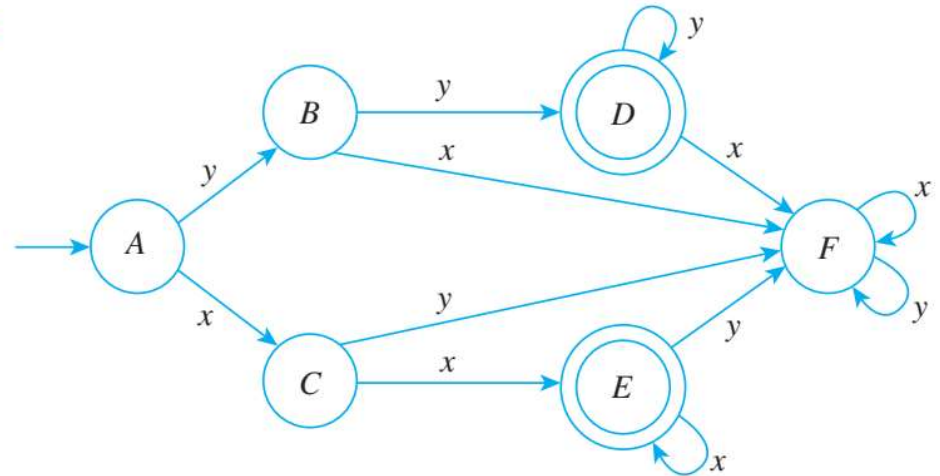
3.



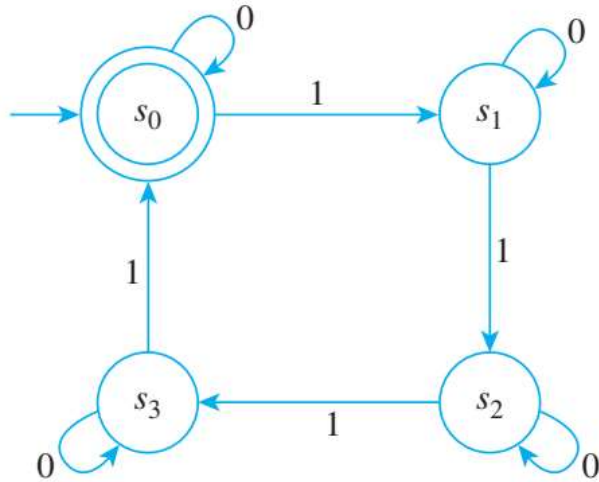
4.



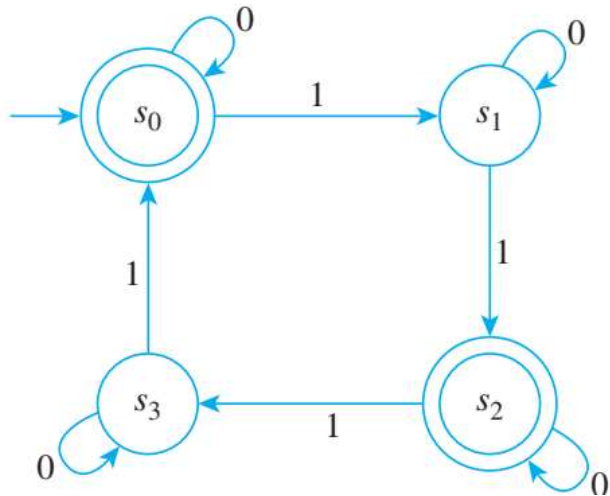
5.



6.



7.



In 8 and 9, a finite-state automaton is given by an annotated next-state table. For each automaton:

- Find its states.
- Find its input symbols.
- Find its initial state.
- Find its accepting states.
- Draw its transition diagram.

8. Next-State Table

		Input	
		0	1
State	→	s_0	s_1
	⊙	s_1	s_2
		s_2	s_1

9. Next-State Table

		Input	
		0	1
State	→	s_0	s_0
	⊙	s_1	s_1
		s_2	s_2
		s_3	s_3